Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**
Образовательная программа **Мобильные и сетевые технологии**
Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №4

По дисциплине «Проектирование и реализация баз данных»

Тема: Запросы на выборку и модификацию данных. Представления. Работа с индексами.

Обучающиеся Миронов Иван Андреевич
Факультет прикладной информатики
Группа K3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватели Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург 2026

1 Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2 Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

3 Выполнение работы

3.1 Наименование БД, вариант

Вариант 8. БД «Аэропорт» (допустимо к выполнению для команды из 2-х студентов)

Описание предметной области: необходимо обеспечить продажу билетов на нужный рейс, при отсутствии билетов (необходимого количества билетов) предложить билет на ближайший рейс.

Рейсы выполняются по расписанию. Но есть рейсы, назначаемые на определенный период или разовые.

Рейс может иметь несколько транзитных посадок (до 3-х).

На каждый рейс формируется экипаж из сотрудников компании, выполняющей рейс. В состав экипажа входят первый и второй пилоты, крю (старший стюард) и стюарды. Необходимо хранить данные о прохождении медосмотра перед рейсом (дата, статус, причина недопуска).

Билет может быть приобретен в кассе или онлайн. К базовой стоимости билета может быть дополнительная плата за выбор места, страховку багажа и т.п. Если билет приобретен в кассе, необходимо знать, в какой. Для каждой кассы известны номер и адрес. Кассы могут располагаться в различных населенных пунктах.

При покупке билета номер места может быть неизвестен пассажиру до регистрации на рейс.

БД должна содержать следующий минимальный набор сведений: Бортовой номер самолета. Тип самолета. Количество мест. Страна. Производитель. Грузоподъемность. Скорость. Дата выпуска. Налѐт в часах. Дата последнего ремонта. Назначение самолета. Расход топлива. Код экипажа. Паспортные данные членов экипажа. Номер рейса. Дата вылета. Время вылета. Аэропорт вылета. Аэропорт назначения. Расстояние. Транзитные посадки (прилет, вылет, аэропорт, время в аэропорту). ФИО пассажира. Паспортные данные. Номер места. Тип места. Цена билета. Касса продажи билета (возможен электронный билет) (номер и адрес).

Дополните состав атрибутов на основе анализа предметной области.

Задание 1.1 (ЛР 2 БД). Выполните инфологическое моделирование базы данных системы. (Ограничения задать самостоятельно.)

Задание 1.2. Создайте логическую модель БД, используя ИЛМ (задание 1.1). Используйте необходимые средства поддержки целостности данных в СУБД.

Задание 2. Создать запросы:

- Определить расчетное время полета по всем маршрутам.
- Определить расход топлива по всем маршрутам.
- Вывести данные о том, сколько свободных мест оставалось в самолетах, совершавших полет по заданному рейсу за вчерашний день.
- Рассчитать убытки компании за счет непроданных билетов за вчерашний день.
- Определить, какой тип самолетов чаще всего летал в заданный аэропорт назначения.
- Вывести список самолетов, “возраст” которых превышает средний “возраст” самолетов этого типа.
- Определить тип самолетов, летающих во все аэропорты назначения.

Задание 3. Создать представление:

- для пассажиров авиакомпании о рейсах в Москву на ближайшую неделю;
- количество самолетов каждого типа, летавшими за последний месяц.

Задание 4. Создать хранимые процедуры:

- Для поиска билетов в заданный пункт назначения.
- Создания новой кассы продажи билетов.

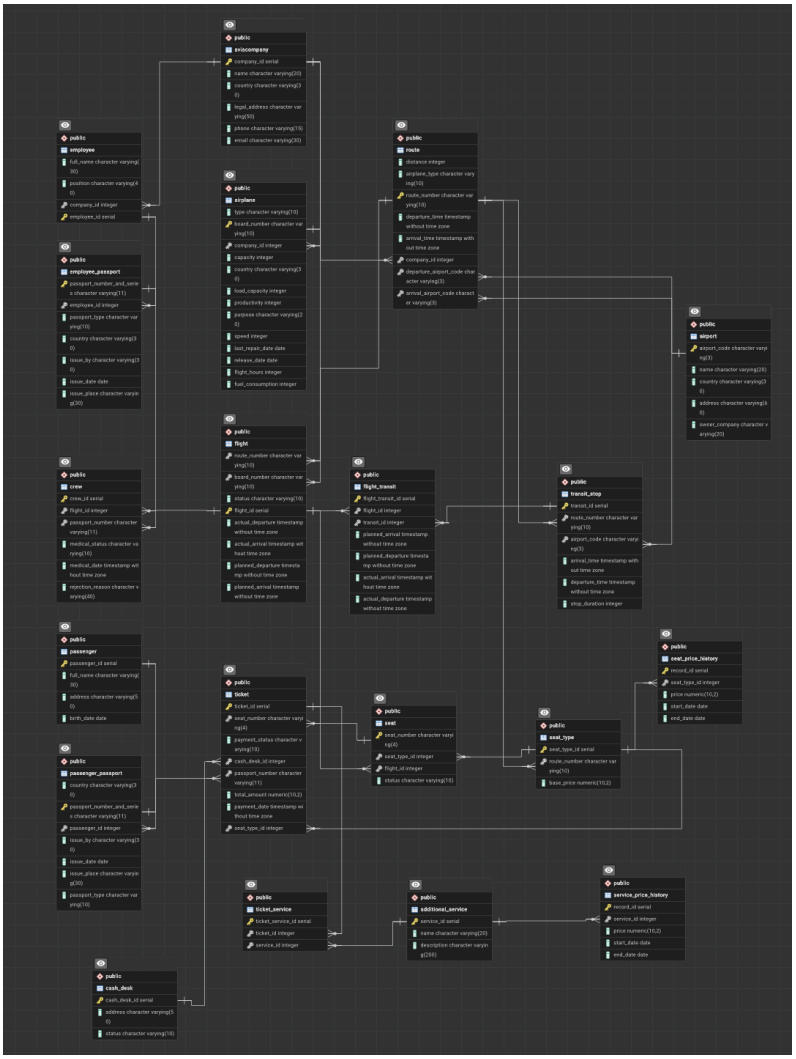
- Определить расход топлива по всем маршрутам за истекший месяц.

Задание 5. Создать необходимые триггеры.

3.2 Схема логической модели базы данных

Для реализации работы была установлена СУБД PostgreSQL версии 18. С помощью графической оболочки pgAdmin 4 была создана база данных lab3_airport.

После чего в Generate ERD была создана диаграмма:



3.4 Выполнение 2 части индивидуального задания

Запрос 1. Определить расчетное время полета по всем маршрутам

Как считается: Разница между временем прибытия и временем отправления

Query Query History AI Assistant

```
SELECT
    route_number,
    departure_time,
    arrival_time,
    (arrival_time - departure_time) AS flight_duration
FROM route;
```

Data Output Messages Notifications

Showing rows: 1 t

	route_number [PK] character varying (10)	departure_time timestamp without time zone	arrival_time timestamp without time zone	flight_duration interval
1	SU-101	2025-01-15 08:00:00	2025-01-15 11:30:00	03:30:00
2	SU-102	2025-01-15 10:00:00	2025-01-15 14:00:00	04:00:00
3	SU-103	2025-01-15 14:00:00	2025-01-15 20:00:00	06:00:00
4	S7-201	2025-01-16 07:00:00	2025-01-16 10:30:00	03:30:00
5	S7-202	2025-01-16 09:00:00	2025-01-16 13:00:00	04:00:00

Запрос 2. Определить расход топлива по всем маршрутам.

Как считается: Произведение расхода самолета на дистанцию

Query Query History AI Assistant

```
SELECT
    r.route_number,
    r.distance,
    a.fuel_consumption,
    (r.distance * a.fuel_consumption) AS total_fuel
FROM route r
JOIN flight f ON r.route_number = f.route_number
JOIN airplane a ON f.board_number = a.board_number;
```

Data Output Messages Notifications

	route_number character varying (10)	distance integer	fuel_consumption integer	total_fuel integer
1	SU-101	2500	2500	6250000
2	SU-102	3200	2400	7680000
3	SU-103	5500	5000	27500000
4	S7-201	2800	4500	12600000

Запрос 3. Вывести данные о том, сколько свободных мест оставалось в самолетах, совершавших полет по заданному рейсу за вчерашний день.

Как считается: через заданный статус возьмем конкретную дату и вычтем 1 день

Query Query History AI Assistant

```sql
SELECT
    f.flight_id,
    COUNT(s.seat_number) AS free_seats
FROM flight f
JOIN seat s ON f.flight_id = s.flight_id
WHERE s.status = 'available'
    AND f.flight_id = 1
AND DATE(f.planned_departure) = DATE '2025-01-16' - INTERVAL '1 day'
GROUP BY f.flight_id;
```

Data Output Messages Notifications

Showir

	flight_id [PK] integer	free_seats bigint
1	1	2

Запрос 4. Рассчитать убытки компании за счет непроданных билетов за вчерашний день.

Как считается: сумма стоимости свободных мест

Query Query History AI Assistant

```sql
SELECT
    f.flight_id,
    COUNT(s.seat_number) AS unsold_seats,
    SUM(st.base_price) AS losses
FROM flight f
JOIN seat s ON f.flight_id = s.flight_id
JOIN seat_type st ON s.seat_type_id = st.seat_type_id
JOIN route r ON f.route_number = r.route_number
WHERE s.status = 'available'
    AND f.flight_id = 3
    AND r.company_id = 1
    AND DATE(f.planned_departure) = DATE '2025-01-16' - INTERVAL '1 day'
GROUP BY f.flight_id;
```

Data Output Messages Notifications

Showing ro

	flight_id [PK] integer	unsold_seats bigint	losses numeric
1	3	4	60000.00

Запрос 5. Определить, какой тип самолетов чаще всего летал в заданный аэропорт назначения.

Как считается: тот, у которого максимальное количество рейсов в заданный аэропорт назначения.

Query Query History AI Assistant

```
SELECT
    a.type,
    COUNT(*) AS flights_count
FROM flight f
JOIN airplane a ON f.board_number = a.board_number
JOIN route r ON f.route_number = r.route_number
WHERE r.arrival_airport_code = 'LED'
GROUP BY a.type
ORDER BY flights_count DESC;
```

Data Output Messages Notifications

	type character varying (10)	flights_count bigint
1	A350	1
2	B737	1

Запрос 6. Вывести список самолетов, “возраст” которых превышает средний “возраст” самолетов этого типа.

Как считается: берутся все самолёты, для каждого считается возраст, потом для каждого типа считается средний возраст, и выбираются те самолёты, у которых возраст больше этого среднего.

Query Query History AI Assistant

```
SELECT
    a1.board_number,
    a1.type,
    (CURRENT_DATE - a1.release_date) AS age
FROM airplane a1
WHERE (CURRENT_DATE - a1.release_date) >
(
    SELECT AVG(CURRENT_DATE - a2.release_date)
    FROM airplane a2
    WHERE a2.type = a1.type
);
```

Data Output Messages Notifications

	board_number [PK] character varying (10)	type character varying (10)	age integer
1	RA-73001	B737	2166
2	A6-E001	B777	3124
3	D-AI001	A320	2560
4	F-HP001	A380	3418
5	RA-73006	B737	2359

Запрос 7. Определить тип самолетов, летающих во все аэропорты назначения.

Как считается: Берутся все рейсы, смотрится, в какие аэропорты летал каждый тип самолета, считается количество разных аэропортов для каждого типа и выбирается тот, у которого их больше всего.

Query Query History AI Assistant

```sql
SELECT a.type
FROM airplane a
JOIN flight f ON a.board_number = f.board_number
JOIN route r ON f.route_number = r.route_number
GROUP BY a.type
HAVING COUNT(DISTINCT r.arrival_airport_code) =
(
    SELECT COUNT(DISTINCT arrival_airport_code)
    FROM route
);
```

Data Output Messages Notifications

SQL

type character varying (10)

3.5 Выполнение 3 части индивидуального задания

Представление 1. Для пассажиров авиакомпании о рейсах в Москву на ближайшую неделю

Создание представления:

Query Query History AI Assistant

```
CREATE VIEW flights_to_moscow AS
SELECT
    f.flight_id,
    f.planned_departure,
    f.planned_arrival,
    r.route_number,
    r.arrival_airport_code
FROM flight f
JOIN route r ON f.route_number = r.route_number
JOIN aviacompany ac ON r.company_id = ac.company_id
WHERE r.arrival_airport_code IN ('SVO', 'DME', 'VKO')
AND ac.name = 'Aeroflot'
AND DATE(f.planned_departure) BETWEEN DATE '2025-01-15' AND DATE '2025-01-15' + INTERVAL '7 day';
```

Data Output Messages Notifications

```
CREATE VIEW

Query returned successfully in 38 msec.
```

Просмотр содержимого:

Query Query History AI Assistant

```
SELECT * FROM public.flights_to_moscow
```

Data Output Messages Notifications

Showing rows: 1 to 1

	flight_id integer	planned_departure timestamp without time zone	planned_arrival timestamp without time zone	route_number character varying (10)	arrival_airport_code character varying (3)
1	11	2025-01-19 12:00:00	2025-01-19 15:30:00	SU-104	VKO

Представление 2. Количество самолетов каждого типа, летавшие за последний месяц.

Создание представления:

Query Query History AI Assistant

```
CREATE VIEW airplane_type_last_month AS
SELECT
    a.type,
    COUNT(f.board_number) AS airplane_count
FROM flight f
JOIN airplane a ON f.board_number = a.board_number
WHERE DATE(f.planned_departure) BETWEEN DATE '2025-01-15' - INTERVAL '1 month' AND DATE '2025-01-20'
GROUP BY a.type;
```

Data Output Messages Notifications

```
CREATE VIEW

Query returned successfully in 35 msec.
```

Просмотр содержимого:

Query Query History AI Assistant

```
SELECT * FROM public.airplane_type_last_month
```

Data Output Messages Notifications

	type character varying (10)	airplane_count bigint
1	A320	2
2	A321	1
3	A330	1
4	A350	1
5	A380	1
6	B737	3
7	B747	1
8	B777	2
9	B787	1

3.6 Запросы на модификацию данных

Запрос 1 (INSERT с подзапросом). Добавить билет для пассажира с минимальным ID.

До:

Query Query History AI Assistant

```
SELECT * FROM public.ticket
ORDER BY ticket_id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	ticket_id [PK] integer	seat_number character varying (4)	payment_status character varying (10)	cash_desk_id integer	passport_number character varying (11)	total_amount numeric (10,2)	payment_date timestamp without time zone	seat_type_id integer
1	1	1A	paid	1	P123456780	5000.00	2025-01-10 14:30:00	1
2	2	1B	paid	1	P123456781	5000.00	2025-01-10 15:00:00	1
3	3	2B	paid	2	P123456782	5000.00	2025-01-11 10:00:00	1
4	4	3A	paid	2	P123456783	6500.00	2025-01-11 11:30:00	2
5	5	3C	paid	3	P123456784	6500.00	2025-01-11 12:00:00	2
6	6	4B	paid	3	P123456785	6500.00	2025-01-12 09:00:00	2
7	7	5C	paid	4	P123456786	15000.00	2025-01-12 10:30:00	3
8	8	8B	paid	5	P123456787	6000.00	2025-01-13 14:00:00	5
9	9	1A	pending	6	P123456788	25000.00	[null]	6
10	10	3A	paid	7	P123456789	10000.00	2025-01-14 16:00:00	8

Выполнение запроса:

Query Query History AI Assistant

```
INSERT INTO ticket (seat_number, payment_status, passport_number, total_amount, seat_type_id)
VALUES (
    '6B',
    'pending',
    (SELECT passport_number_and_series
     FROM passenger_passport
     WHERE passenger_id = (
         SELECT MIN(passenger_id) FROM passenger_passport
     )),
    5000,
    (SELECT MIN(seat_type_id) FROM seat_type)
);
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 42 msec.
```

После выполнения добавилась еще одна строка:

Query Query History AI Assistant

```
SELECT * FROM public.ticket
ORDER BY ticket_id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

	ticket_id [PK] integer	seat_number character varying (4)	payment_status character varying (10)	cash_desk_id integer	passport_number character varying (11)	total_amount numeric (10,2)	payment_date timestamp without time zone	seat_type_id integer
1	1	1A	paid	1	P123456780	5000.00	2025-01-10 14:30:00	1
2	2	1B	paid	1	P123456781	5000.00	2025-01-10 15:00:00	1
3	3	2B	paid	2	P123456782	5000.00	2025-01-11 10:00:00	1
4	4	3A	paid	2	P123456783	6500.00	2025-01-11 11:30:00	2
5	5	3C	paid	3	P123456784	6500.00	2025-01-11 12:00:00	2
6	6	4B	paid	3	P123456785	6500.00	2025-01-12 09:00:00	2
7	7	5C	paid	4	P123456786	15000.00	2025-01-12 10:30:00	3
8	8	8B	paid	5	P123456787	6000.00	2025-01-13 14:00:00	5
9	9	1A	pending	6	P123456788	25000.00	[null]	6
10	10	3A	paid	7	P123456789	10000.00	2025-01-14 16:00:00	8
11	12	6B	pending	[null]	P123456780	5000.00	[null]	1

Из таблицы passanger_passport видно что пассажиру с наименьшим id = 1, соответствуют паспортные данные.

1	Russia	P123456780	1	MVD Russia	2020-01-15	Moscow	Internat

Запрос 2 (UPDATE с подзапросом). Обновить статус билета на "paid" для самого дорогого билета.

До выполнения видно что для билета с id = 9 статус стоит pending.

Query Query History AI Assistant

```
SELECT * FROM public.ticket
ORDER BY ticket_id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

	ticket_id [PK] integer	seat_number character varying (4)	payment_status character varying (10)	cash_desk_id integer	passport_number character varying (11)	total_amount numeric (10,2)	payment_date timestamp without time zone	seat_type_id integer
1	1	1A	paid	1	P123456780	5000.00	2025-01-10 14:30:00	1
2	2	1B	paid	1	P123456781	5000.00	2025-01-10 15:00:00	1
3	3	2B	paid	2	P123456782	5000.00	2025-01-11 10:00:00	1
4	4	3A	paid	2	P123456783	6500.00	2025-01-11 11:30:00	2
5	5	3C	paid	3	P123456784	6500.00	2025-01-11 12:00:00	2
6	6	4B	paid	3	P123456785	6500.00	2025-01-12 09:00:00	2
7	7	5C	paid	4	P123456786	15000.00	2025-01-12 10:30:00	3
8	8	8B	paid	5	P123456787	6000.00	2025-01-13 14:00:00	5
9	9	1A	pending	6	P123456788	25000.00	[null]	6
10	10	3A	paid	7	P123456789	10000.00	2025-01-14 16:00:00	8
11	12	6B	pending	[null]	P123456780	5000.00	[null]	1

Выполнение запроса:

Query Query History AI Assistant

```
UPDATE ticket
SET payment_status = 'paid'
WHERE total_amount = (
    SELECT MAX(total_amount) FROM ticket
);
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 52 msec.
```

После выполнения у этого билета поменялся статус.

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

	ticket_id [PK] integer	seat_number character varying (4)	payment_status character varying (10)	cash_desk_id integer	passport_number character varying (11)	total_amount numeric (10,2)	payment_date timestamp without time zone	seat_type_id integer
1	1	1A	paid	1	P123456780	5000.00	2025-01-10 14:30:00	1
2	2	1B	paid	1	P123456781	5000.00	2025-01-10 15:00:00	1
3	3	2B	paid	2	P123456782	5000.00	2025-01-11 10:00:00	1
4	4	3A	paid	2	P123456783	6500.00	2025-01-11 11:30:00	2
5	5	3C	paid	3	P123456784	6500.00	2025-01-11 12:00:00	2
6	6	4B	paid	3	P123456785	6500.00	2025-01-12 09:00:00	2
7	7	5C	paid	4	P123456786	15000.00	2025-01-12 10:30:00	3
8	8	8B	paid	5	P123456787	6000.00	2025-01-13 14:00:00	5
9	9	1A	paid	6	P123456788	25000.00	[null]	6
10	10	3A	paid	7	P123456789	10000.00	2025-01-14 16:00:00	8
11	12	6B	pending	[null]	P123456780	5000.00	[null]	1

Запрос 3 (DELETE с подзапросом). Удаление записей экипажа, назначенного на завершенные рейсы

Data Output Messages Notifications

Showing rows: 1 to 20 Page N

	crew_id [PK] integer	flight_id integer	passport_number character varying (11)	medical_status character varying (10)	medical_date timestamp without time zone	rejection_reason character varying (40)
1	1	1	12345678901	approved	2024-12-01 10:00:00	[null]
2	2	1	12345678902	approved	2024-12-01 10:00:00	[null]
3	3	2	12345678903	approved	2024-12-02 10:00:00	[null]
4	4	2	12345678904	approved	2024-12-02 10:00:00	[null]
5	5	3	12345678905	approved	2024-12-03 10:00:00	[null]
6	6	3	12345678906	approved	2024-12-03 10:00:00	[null]
7	7	4	12345678907	approved	2024-12-04 10:00:00	[null]
8	8	4	12345678908	approved	2024-12-04 10:00:00	[null]
9	9	5	12345678909	approved	2024-12-05 10:00:00	[null]
10	10	5	12345678910	approved	2024-12-05 10:00:00	[null]
11	11	6	A123456789	approved	2024-12-06 10:00:00	[null]
12	12	6	B123456789	approved	2024-12-06 10:00:00	[null]
13	13	7	C123456789	approved	2024-12-07 10:00:00	[null]
14	14	7	D123456789	approved	2024-12-07 10:00:00	[null]
15	15	8	E123456789	approved	2024-12-08 10:00:00	[null]
16	16	8	F123456789	approved	2024-12-08 10:00:00	[null]
17	17	9	G123456789	approved	2024-12-09 10:00:00	[null]
18	18	9	H123456789	approved	2024-12-09 10:00:00	[null]
19	19	10	12345678911	approved	2024-12-10 10:00:00	[null]
20	20	10	12345678912	approved	2024-12-10 10:00:00	[null]

Выполнение запроса:

Query Query History AI Assistant

```
DELETE FROM crew
WHERE flight_id IN (
    SELECT flight_id
    FROM flight
    WHERE status = 'completed'
);
```

Data Output Messages Notifications

```
DELETE 4

Query returned successfully in 95 msec.
```

После выполнения у нас стало на 4 строки меньше.

Showing rows: 1 to 16

	crew_id [PK] integer	flight_id integer	passport_number character varying (11)	medical_status character varying (10)	medical_date timestamp without time zone	rejection_reason character varying (40)
1	5	3	12345678905	approved	2024-12-03 10:00:00	[null]
2	6	3	12345678906	approved	2024-12-03 10:00:00	[null]
3	7	4	12345678907	approved	2024-12-04 10:00:00	[null]
4	8	4	12345678908	approved	2024-12-04 10:00:00	[null]
5	9	5	12345678909	approved	2024-12-05 10:00:00	[null]
6	10	5	12345678910	approved	2024-12-05 10:00:00	[null]
7	11	6	A123456789	approved	2024-12-06 10:00:00	[null]
8	12	6	B123456789	approved	2024-12-06 10:00:00	[null]
9	13	7	C123456789	approved	2024-12-07 10:00:00	[null]
10	14	7	D123456789	approved	2024-12-07 10:00:00	[null]
11	15	8	E123456789	approved	2024-12-08 10:00:00	[null]
12	16	8	F123456789	approved	2024-12-08 10:00:00	[null]
13	17	9	G123456789	approved	2024-12-09 10:00:00	[null]
14	18	9	H123456789	approved	2024-12-09 10:00:00	[null]
15	19	10	12345678911	approved	2024-12-10 10:00:00	[null]
16	20	10	12345678912	approved	2024-12-10 10:00:00	[null]

3.7 Создание индексов

В таблицу абонентов было добавлено 1000 записей, чтобы заметить разницу во времени выполнения запроса с помощью индекса и без него. В качестве тестового запроса сделаем выборку мест, доступных к покупке:

Query Query History AI Assistant

```
EXPLAIN ANALYZE
SELECT * FROM seat
WHERE status = 'available';
```

Data Output Messages Notifications

	QUERY PLAN text
1	Seq Scan on seat (cost=0.00..10.99 rows=383 width=20) (actual time=0.027..0.099 rows=383.00 loop…
2	Filter: ((status)::text = 'available'::text)
3	Rows Removed by Filter: 176
4	Buffers: shared hit=4
5	Planning:
6	Buffers: shared hit=23
7	Planning Time: 0.750 ms
8	Execution Time: 0.139 ms

Благодаря использованию команды EXPLAIN ANALYZE, видим что:

Результаты без использования индекса:

- Planning Time: 0.750 ms
- Execution Time: 0.139 ms

Далее был создан индекс для поля status из таблицы seat:

Query Query History AI Assistant

```
1 CREATE INDEX idx_seat_status
2 ON seat(status);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 40 msec.
```

Теперь еще раз выполняем запрос:

Query Query History AI Assistant

```
1 EXPLAIN ANALYZE
2 SELECT * FROM seat
3 WHERE status = 'available';
```

Data Output Messages Notifications

Showing r

	QUERY PLAN text
1	Seq Scan on seat (cost=0.00..10.99 rows=383 width=20) (actual time=0.016..0.079 rows=383.00 loop...
2	Filter: ((status)::text = 'available'::text)
3	Rows Removed by Filter: 176
4	Buffers: shared hit=4
5	Planning Time: 0.080 ms
6	Execution Time: 0.109 ms

И видим что с использованием индекса:

- Planning Time: 0.080 ms
- Execution Time: 0.109 ms

Далее удаляем созданный индекс.

Query Query History AI Assistant

```
1 DROP INDEX idx_seat_status;
```

Data Output Messages Notifications

```
DROP INDEX

Query returned successfully in 50 msec.
```

Из полученных данных видно, что время выполнения запроса (Execution Time) практически не изменилось, так как объем данных в таблице относительно небольшой.

Но можно заметить уменьшение времени планирования (Planning Time) после создания индекса. Потому что планировщик PostgreSQL получил дополнительную возможность выбора более эффективного плана выполнения запроса благодаря наличию индекса.

Выводы

В ходе выполнения лабораторной работы были созданы запросы на выборку данных, построены представления, составлены запросы на модификацию данных с использованием подзапросов, изучено использование команды EXPLAIN ANALYZE, а также исследовано влияние индексов на время выполнения запросов.

В результате выполнения работы были приобретены практические навыки:

- составления запросов на выборку данных к базе данных PostgreSQL;
- создания и использования представлений;
- применения подзапросов в запросах на модификацию данных;
- анализа истории запросов и графического плана выполнения;
- создания простых и составных индексов;
- сравнительного анализа времени выполнения запросов без индексов и с индексами.